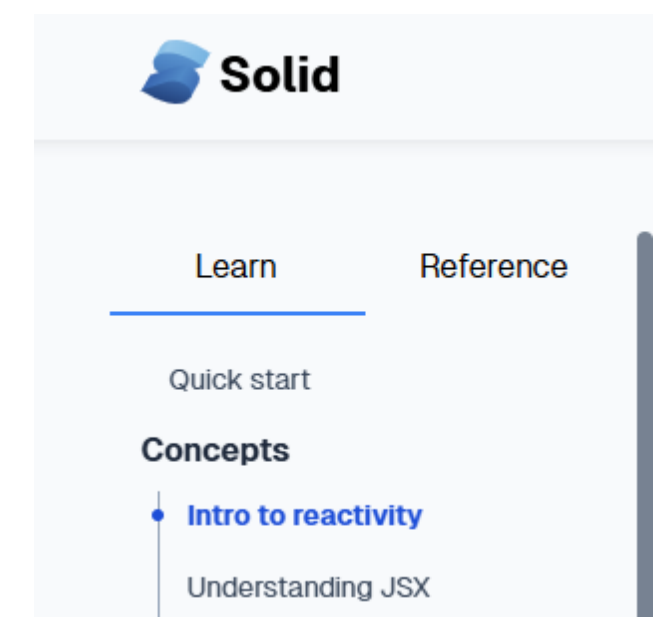
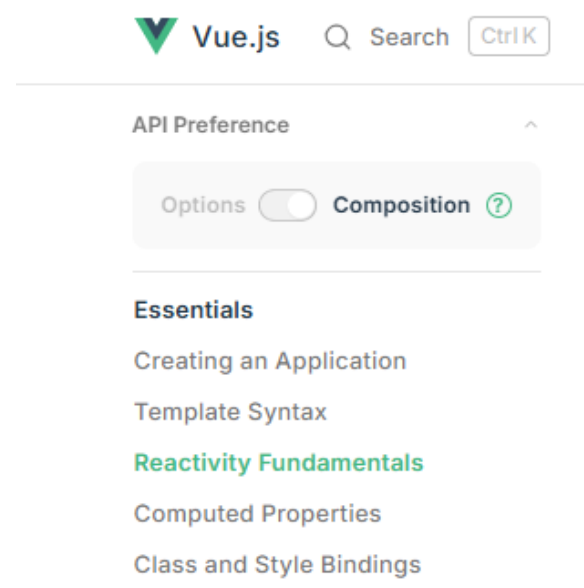
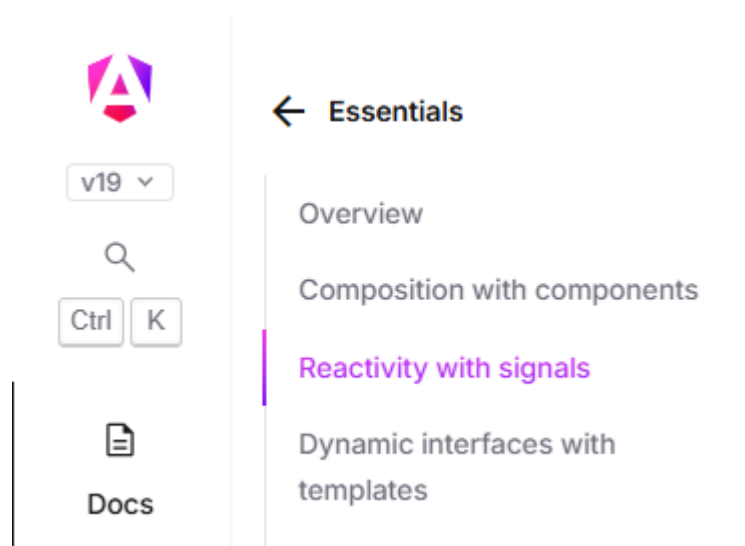


Reaktivní programování v kontextu webových aplikací

Proč se zajímat o reaktivitu





Michal Štrajt



Michal Štrajt

- Technický leader v Edhouse





Michal Štrajt

- Technický leader v Edhouse
- Frontend vývojář



Michal Štrajt

- Technický leader v Edhouse
- Frontend vývojář
- 7 let zkušeností s frontendem



Michal Štrajt

- Technický leader v Edhouse
- Frontend vývojář
- 7 let zkušeností s frontendem
- Programátor, otec, improvizátor

Imperativní vs deklarativní programování

Imperativní programování

```
// imperativní (computedPrice může být nahrazen po deklaraci)  
let computedPrice: number;
```

```
// imperativní (computedPrice může být nahrazen po deklaraci)  
let computedPrice: number;
```

```
// tady nastavuju poprvé  
function setup(): void {  
    computedPrice = 0;  
}
```



```
// imperativní (computedPrice může být nahrazen po deklaraci)
let computedPrice: number;

// tedy nastavuju poprvé
function setup(): void {
  computedPrice = 0;
}

// koukejte, tedy ho nastavuju znovu
async function loadComputedPrice(): Promise<void> {
  const price = await loadPrice();
  const amount = getCurrentAmount();
  computedPrice = price * amount;
}
```

```
// imperativní (computedPrice může být nahrazen po deklaraci)
let computedPrice: number;

// tedy nastavuju poprvé
function setup(): void {
  computedPrice = 0;
}

// koukejte, tedy ho nastavuju znovu
async function loadComputedPrice(): Promise<void> {
  const price = await loadPrice();
  const amount = getCurrentAmount();
  computedPrice = price * amount;
}

// překvapení, tedy ho nastavuju ještě jednou
function addToCart(): void {
  const price = getCurrentPrice();
  const amount = getCurrentAmount() + 1;
  computedPrice = price * amount;
}
```

```
// imperativní (computedPrice může být nahrazen po deklaraci)
let computedPrice: number;
let isExpensive: number;

// tedy nastavuju poprvé
function setup(): void {
  computedPrice = 0;
  setIsExpensive();
}

// koukejte, tedy ho nastavuju znovu
async function loadComputedPrice(): Promise<void> {
  const price = await loadPrice();
  const amount = getCurrentAmount();
  computedPrice = price * amount;
  setIsExpensive();
}

// překvapení, tedy ho nastavuju ještě jednou
function addToCart(): void {
  const price = getCurrentPrice();
  const amount = getCurrentAmount() + 1;
  computedPrice = price * amount;
  setIsExpensive();
}

function setIsExpensive(): void {
  isExpensive = computedPrice > 999;
}
```

Deklarativní programování

```
// deklarativní (computedPrice nemůže být nahrazen po deklaraci)  
const initialPrice = 0;  
const initialAmount = 0;
```



```
// deklarativní (computedPrice nemůže být nahrazen po deklaraci)
```

```
const initialPrice = 0;
```

```
const initialAmount = 0;
```

```
// deklarativní (price nemůže být nahrazen po deklaraci, ale jeho hodnota se může měnit)
```

```
const price = someDeclarativePrimitive(initialPrice);
```

```
const amount = someDeclarativePrimitive(initialAmount);
```

```
// deklarativní (computedPrice nemůže být nahrazen po deklaraci)
const initialPrice = 0;
const initialAmount = 0;

// deklarativní (price nemůže být nahrazen po deklaraci, ale jeho hodnota se může měnit)
const price = someDeclarativePrimitive(initialPrice);
const amount = someDeclarativePrimitive(initialAmount);

// deklarativní (veškeré chování computedPrice price je popsáno v jeho deklaraci)
const computedPrice = someDeclarativeComputation(() => price.value() * amount.value());
```

```
// deklarativní (computedPrice nemůže být nahrazen po deklaraci)
const initialPrice = 0;
const initialAmount = 0;

// deklarativní (price nemůže být nahrazen po deklaraci, ale jeho hodnota se může měnit)
const price = someDeclarativePrimitive(initialPrice);
const amount = someDeclarativePrimitive(initialAmount);

// deklarativní (veškeré chování computedPrice price je popsáno v jeho deklaraci)
const computedPrice = someDeclarativeComputation(() => price.value() * amount.value());

const isExpensive = someDeclarativeComputation(() => computedPrice.value() > 999);
```

Imperativní × Deklarativní × Reaktivní

Imperativní × Deklarativní × Reaktivní

Imperativní programování

- Popisuje JAK se má kód vykonávat
- Volá funkce a nastavuje proměnné

Imperativní × Deklarativní × Reaktivní

Imperativní programování

- Popisuje JAK se má kód vykonávat
- Volá funkce a nastavuje proměnné

Deklarativní programování

- Popisuje CO má kód vykonávat
- Jako programátor neřeším JAK se kód vykonat, to za mě řeší knihovna

Imperativní × Deklarativní × Reaktivní

Imperativní programování

- Popisuje JAK se má kód vykonávat
- Volá funkce a nastavuje proměnné

Deklarativní programování

- Popisuje CO má kód vykonávat
- Jako programátor neřeším JAK se kód vykonat, to za mě řeší knihovna

Reaktivní programování

- Popisuje CO má kód vykonávat v čase
- Typ deklarativního programování, kde deklarativní primitiva mohou měnit hodnoty v čase

Totéž, ale místo kódu obrázky

Imperativní a deklarativní programování graficky

Imperativní a deklarativní programování graficky

Imperativní a deklarativní programování graficky


```
<div>
  <span>Doprava zdarma:</span>
  <span>{{ isExpensive }}</span>
</div>
```

```
<div>  
  <span>Doprava zdarma:</span>  
  <span>{{ isExpensive }}</span>  
</div>
```

Kvíz

Kvíz

```
var a = 4;  
var b = 2;  
var c = a + b;  
b = 10;  
console.log(c);
```

Kvíz

```
var a = 4;  
var b = 2;  
var c = a + b;  
b = 10;  
console.log(c);
```

- 6

Kvíz

```
var a = 4;  
var b = 2;  
var c = a + b;  
b = 10;  
console.log(c);
```

- 6
- 14

Kvíz

```
var a = 4;  
var b = 2;  
var c = a + b;  
b = 10;  
console.log(c);
```

- 6
- 14
- Něco jiného

Kvíz

```
var a = 4;  
var b = 2;  
var c = a + b;  
b = 10;  
console.log(c);
```

- 6
- 14
- Něco jiného



Kvíz

```
var a = 4;  
var b = 2;  
var c = a + b;  
b = 10;  
console.log(c);
```

- 6
- 14
- Něco jiného



Signal

```
const price = signal(0);  
const amount = signal(0);
```



```
const price = signal(0);
const amount = signal(0);

// imperativní
amount.set(42);

// deklarativní
<input type="number" [(value)]="price" />
```

```
const price = signal(0);
const amount = signal(0);

const computedPrice = computed(() => price() * amount());

// logs 0
console.log(computedPrice());

// imperativní nastavení
price.set(6)
amount.set(7);

// logs 42
console.log(computedPrice());
```

```
const price = signal(0);
const amount = signal(0);

const computedPrice = computed(() => price() * amount());

<input type="number" [(value)]= "price" />
<input type="number" [(value)]= "amount" />
<div>{{ computedPrice() }}</div>
```

Signal

Signal

- Deklarativní popis dat

Signal

- Deklarativní popis dat
- Jedna hodnota, která se mění v čase

Signal

- Deklarativní popis dat
- Jedna hodnota, která se mění v čase
- Umožňuje kompozici více signálu do jednoho

Observable

Observer pattern

Observer pattern

```
const timer = new Observable(subscriber => {  
  const interval = setInterval(() => {  
    // tedy notifikuje subscribera  
    subscriber.next()  
  }, 1000)  
  
  return () => {  
    clearInterval(interval)  
  }  
})
```

```
const timer = new Observable(subscriber => {
  const interval = setInterval(() => {
    subscriber.next()
  }, 1000)

  return () => {
    clearInterval(interval)
  }
})

// nezačíná nic zajímavého, observable nic nedělá
console.log(timer);
```

```
const timer = new Observable(subscriber => {
  const interval = setInterval(() => {
    subscriber.next()
  }, 1000)

  return () => {
    clearInterval(interval)
  }
})

// nezačíná nic zajímavého, observable nic nedělá
console.log(timer);

const subscription1 = timer.subscribe(() => {
  // pokaždé, když timer notifikuje, zloguje aktuální datum
  console.log(new Date())
})

const subscription2 = timer.subscribe(() => {
  // pokaždé, když timer notifikuje, zloguje "Hello world"
  console.log('Hello world')
})

// nezačíná nic zajímavého
console.log(timer)
```



```
const timer = new Observable(subscriber => {
  const interval = setInterval(() => {
    subscriber.next()
  }, 1000)

  return () => {
    clearInterval(interval)
  }
})

// nezačíná nic zajímavého, observable nic nedělá
console.log(timer);

const subscription1 = timer.subscribe(() => {
  // pokaždé, když timer notifikuje, zloguje aktuální datum
  console.log(new Date())
})

const subscription2 = timer.subscribe(() => {
  // pokaždé, když timer notifikuje, zloguje "Hello world"
  console.log('Hello world')
})

// nezačíná nic zajímavého
console.log(timer)

// až notifikace nepotřebuju, je potřeba uklidit
subscription1.unsubscribe();
subscription2.unsubscribe();
```

Observable

Observable

- Deklarativní popis událostí

Observable

- Deklarativní popis událostí
- Kolekce eventů nebo hodnot v čase

Observable

- Deklarativní popis událostí
- Kolekce eventů nebo hodnot v čase
- Umožňuje kompozici více observable do jednoho

Signal

Observable

Signal

Jedna hodnota měnící se v čase

Observable

Kolekce událostí, které nastávají v čase

Signal

Jedna hodnota měnící se v čase
Výhodný pro synchronní data

Observable

Kolekce událostí, které nastávají v čase
Výhodný pro asynchronní data

Signal

Jedna hodnota měnící se v čase
Výhodný pro synchronní data
Jednoduché použití

Observable

Kolekce událostí, které nastávají v čase
Výhodný pro asynchronní data
Náročné na použití

Signal

Jedna hodnota měnící se v čase

Výhodný pro synchronní data

Jednoduché použití

Náročný na implementaci

Observable

Kolekce událostí, které nastávají v čase

Výhodný pro asynchronní data

Náročné na použití

Jednoduchý na implementaci

Signal

Jedna hodnota měnící se v čase

Výhodný pro synchronní data

Jednoduché použití

Náročný na implementaci

Jednoduchý převod na observable

Observable

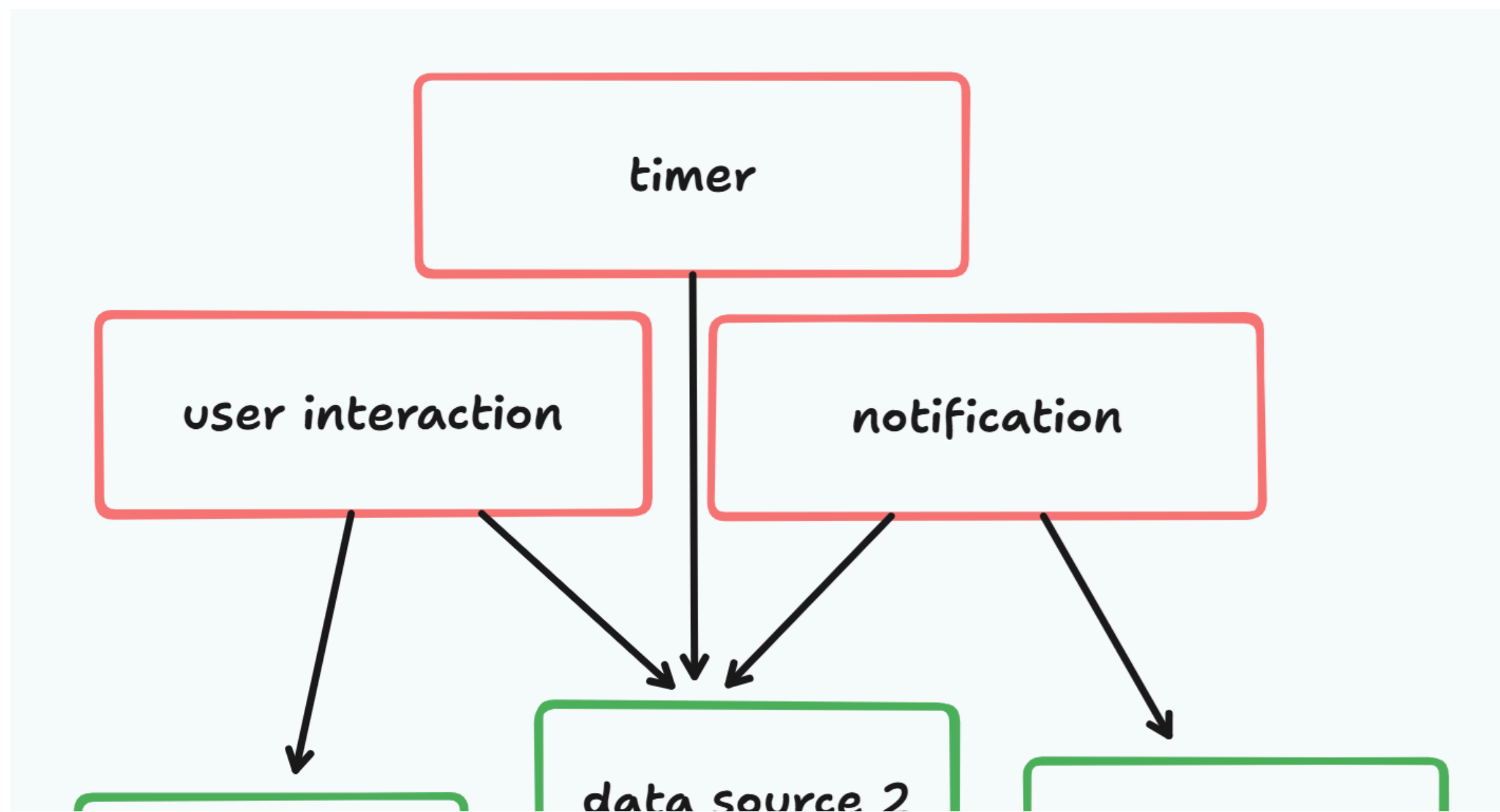
Kolekce událostí, které nastávají v čase

Výhodný pro asynchronní data

Náročné na použití

Jednoduchý na implementaci

Jednoduchý převod na signal



```
// observe klikání na tlačítko  
const buttonClick = buttonElement.when('click');
```

```
// observe klikání na tlačítko
const buttonClick = buttonElement.when('click');
// observable po každém kliku načte data
const loadedPrice = buttonClick.switchMap(() => loadPriceData());
```

```
// observe klikání na tlačítko
const buttonClick = buttonElement.when('click');
// observable po každém kliku načte data
const loadedPrice = buttonClick.switchMap(() => loadPriceData());
// převod price na signal
const price = toSignal(loadedPrice);
```

```
// observe klikání na tlačítko
const buttonClick = buttonElement.when('click');
// observable po každém kliku načte data
const loadedPrice = buttonClick.switchMap(() => loadPriceData());
// převod price na signal
const price = toSignal(loadedPrice);
// signal držíci objednaných kusů
const amount = singal(0);
```



```
// observace klikání na tlačítko
const buttonClick = buttonElement.when('click');
// observable po každém kliku načte data
const loadedPrice = buttonClick.switchMap(() => loadPriceData());
// převod price na signal
const price = toSignal(loadedPrice);
// signal držíci objednaných kusů
const amount = signal(0);
// konečná cena
const totalPrice = computed(() => price() * amount());
// má uživatel nárok na slevu?
const isExpensive = computed(() => totalPrice() > 999);
```

```
// observace klikání na tlačítko
const buttonClick = buttonElement.when('click');
// observable po každém kliku načte data
const loadedPrice = buttonClick.switchMap(() => loadPriceData());
// převod price na signal
const price = toSignal(loadedPrice);
// signal držící objednaných kusů
const amount = singal(0);
// konečná cena
const totalPrice = computed(() => price() * amount());
// má uživatel nárok na slevu?
const isExpensive = computed(() => totalPrice() > 999);

<button #buttonElement>Reload prices</button>
<input type="number" [(value)]="amount" />
<div>
  <span>Is expensive?</span>
  <span>{{ isExpensive() }}</span>
</div>
```

A co performance?

Vážně potřebuji framework?

Vážně potřebuji framework?

```
async function* timer(limit) {  
  for (let i = 0; i < limit; i++) {  
    await new Promise(resolve => setTimeout(resolve, 1000));  
    yield i;  
  }  
}
```

```
Observable.from(timer(10)).subscribe(console.log)
```

Shrnutí

Shrnutí

- Deklarativní kód popisuje CO, ne JAK

Shrnutí

- Deklarativní kód popisuje CO, ne JAK
- Reaktivní kód je deklarativní kód, který se vyvíjí v čase

Shrnutí

- Deklarativní kód popisuje CO, ne JAK
- Reaktivní kód je deklarativní kód, který se vyvíjí v čase
- Reaktivní kód je výhodný pro popis UI

Shrnutí

- Deklarativní kód popisuje CO, ne JAK
- Reaktivní kód je deklarativní kód, který se vyvíjí v čase
- Reaktivní kód je výhodný pro popis UI
- V reaktivním kódu odvozujete stav místo nastavování stavu

Shrnutí

- Deklarativní kód popisuje CO, ne JAK
- Reaktivní kód je deklarativní kód, který se vyvíjí v čase
- Reaktivní kód je výhodný pro popis UI
- V reaktivním kódu odvozujete stav místo nastavování stavu
- V JavaScriptu existuje mnoho reaktivních knihoven

Shrnutí

- Deklarativní kód popisuje CO, ne JAK
- Reaktivní kód je deklarativní kód, který se vyvíjí v čase
- Reaktivní kód je výhodný pro popis UI
- V reaktivním kódu odvozujete stav místo nastavování stavu
- V JavaScriptu existuje mnoho reaktivních knihoven
- V budoucnu snad reaktivita bude součástí JavaScriptu

Děkuji za pozornost